



UNIDAD II • TEORIA

Spring Security: fundamentos

La teoria antes de la practica: que problema resuelve, como esta construido por dentro y con que vocabulario hablaremos del resto de la unidad.

Autenticacion • Autorizacion • Cadena de filtros • Sesion vs token

UTEQ • Ingenieria de Software • Dr. Gleiston Guerrero Ulloa, Ph.D.

Que es Spring Security

Es el marco de trabajo estandar para anadir seguridad a aplicaciones Spring. Se encarga de dos cosas: comprobar quien hace una peticion y decidir si tiene permiso. No reinventa la seguridad de Java; se integra con el servlet container y con Spring.



Autenticacion

Verifica la identidad de quien pide.



Autorizacion

Decide que puede hacer cada usuario.



Cadena de filtros

Intercepta cada peticion HTTP.



Proteccion integrada

CSRF, cabeceras, sesiones, cifrado.

Lo que veremos en esta teoria



Autenticacion vs autorizacion

Las dos preguntas de la seguridad.



El contexto de seguridad

Quien soy, durante la peticion.



Usuarios y contraseñas

UserDetailsService y PasswordEncoder.



Sesion vs token

Con estado frente a sin estado.



La cadena de filtros

El corazon arquitectonico.



Quien autentica

AuthenticationManager y proveedores.



Como se autoriza

Roles, authorities, URL y metodo.



Spring Security 6 y 7

Que cambio respecto al pasado.

Los dos pilares de la seguridad



Autenticacion

Quien eres.

Se comprueba la identidad con credenciales (usuario y contraseña, token, certificado...). El resultado es un objeto **Authentication** que representa al usuario ya identificado.



Autorizacion

Que puedes hacer.

Ya identificado, se decide si tiene permiso para un recurso o accion, comparando sus **authorities** (roles o permisos) con las reglas configuradas.

Primero autenticacion, despues autorizacion. Sin saber quien eres, no se puede decidir que puedes hacer.

Principios de diseno de Spring Security



Seguro por defecto

Al anadirlo, todo queda protegido; tu abres lo necesario.



Defensa en profundidad

Varias capas: por URL, por metodo, por dominio.



Extensible

Cada pieza (filtros, proveedores) se puede sustituir.



Integrado, no acoplado

Vive como filtros del servlet; no invade tu logica.

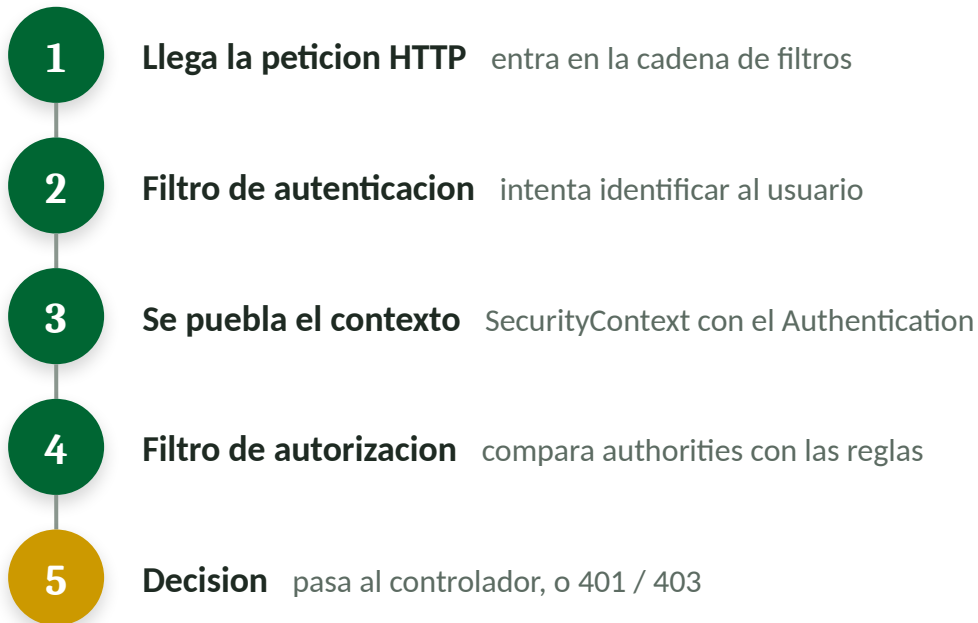
El corazon: la cadena de filtros

Spring Security se inserta en el contenedor de servlets como un filtro. Toda peticion pasa por una cadena de filtros antes de llegar a tu controlador.



Dentro de la SecurityFilterChain hay filtros en orden fijo: uno valida CSRF, otro la autentificacion (por ejemplo nuestro filtro JWT), otro la autorizacion. Cada filtro hace su parte y pasa la peticion al siguiente, o la corta con un 401 / 403.

Como viaja una peticion



El contexto: quien soy en esta peticion

SecurityContextHolder

SecurityContext

Authentication

principal el usuario (UserDetails)

credentials la contraseña (se limpia)

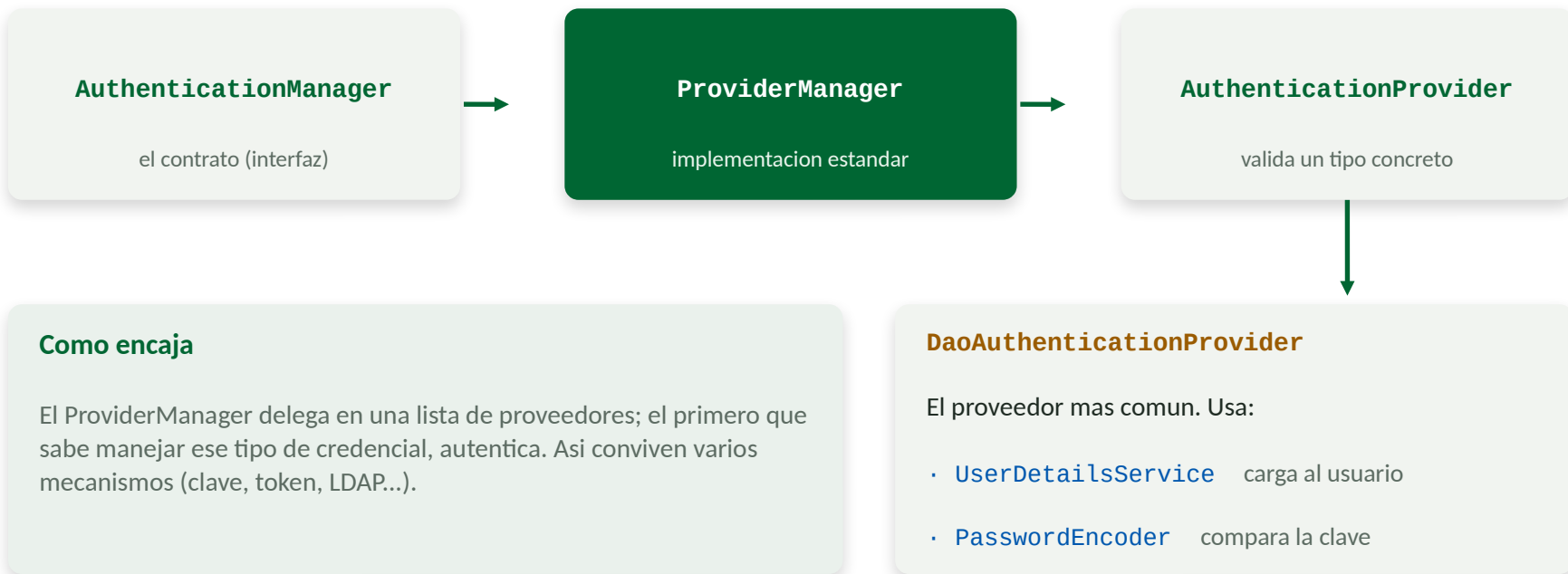
authorities roles y permisos

Por que importa

Tras autenticar, el usuario vive aqui durante la peticion. Cualquier capa puede preguntar quien es el usuario actual.

Por defecto se guarda por hilo (ThreadLocal): cada peticion tiene su propio contexto.

Quien hace el trabajo de autenticar



Como conoce Spring a los usuarios



UserDetails

La vista que Spring tiene de un usuario. **Expone lo imprescindible:**

- getUsername()
- getPassword() (hash)
- getAuthorities()
- cuenta activa / no bloqueada



UserDetailsService

El puente a tu almacen de usuarios. **Un solo metodo:**

`loadUserByUsername(String)`

Lo implementas tu para leer de tu base de datos y devolver un UserDetails. Es el punto donde tu dominio se conecta con Spring Security.

Nunca en texto plano: PasswordEncoder

Regla absoluta: una contraseña jamás se guarda ni se compara en texto plano. Se almacena su **hash**: una huella irreversible. Al iniciar sesión, se vuelve a calcular el hash y se comparan los hashes.



Hash irreversible

Del hash no se recupera la clave original.



Con sal (salt)

Cada clave lleva un valor aleatorio: dos claves iguales dan hashes distintos.



Coste configurable

BCrypt es lento a propósito: frena los ataques de fuerza bruta.

Spring Security usa BCrypt por defecto a través de la interfaz PasswordEncoder.

Roles y authorities: como se decide

Tras autenticar, el usuario tiene una lista de authorities (cadenas que representan permisos). La autorizacion compara esas authorities con lo que exige cada regla.

Rol

Una categoria amplia de usuario. Por convencion lleva el prefijo **ROLE_**

```
hasRole("ADMIN")
```

 busca ROLE_ADMIN.

Authority (permiso)

Un permiso fino y concreto, sin prefijo. Util para acciones especificas

```
hasAuthority("READ_NOTAS")
```

Un rol es, en el fondo, una authority con prefijo.

Internamente todo son authorities; el rol es solo una convencion de nombres con ROLE_.

Dos lugares donde aplicar reglas



Por URL

En la SecurityFilterChain. Filtra pronto, por ruta y metodo HTTP, antes de entrar al controlador.

```
requestMatchers("/admin/**")  
    .hasRole("ADMIN")
```



Por metodo

Sobre metodos concretos, con anotaciones. Regla fina, junto a la logica.

```
@PreAuthorize("hasRole('ADMIN')")  
public void eliminar(...) { }
```

No se excluyen: usar ambas es defensa en profundidad.

Con estado (sesion) vs sin estado (token)



Con estado: sesion

El servidor guarda la sesion en memoria y entrega una cookie. En cada peticion, la cookie identifica la sesion.

+ Simple, revocacion inmediata

- Guarda estado; cuesta escalar



Sin estado: token (JWT)

El cliente guarda un token autocontenido y lo envia en cada peticion. El servidor no guarda nada.

+ Escala; ideal para APIs y SPAs

- Revocar antes de expirar es dificil

Una API REST se disena sin estado; por eso elegimos token (JWT) en la practica.

CSRF y CORS: que son



CSRF

Falsificación de petición entre sitios. Un sitio malicioso provoca que tu navegador envíe una petición con tu cookie de sesión sin que lo sepas.

Importa con sesiones + cookies. Con token en cabecera no aplica: por eso en APIs JWT se suele desactivar.



CORS

Compartición de recursos entre orígenes. Política del navegador que decide si un sitio puede llamar a una API en otro origen.

Importa cuando el cliente (Angular) está en otro servidor. Se configura permitiendo orígenes concretos.

No son lo mismo: CSRF es un ataque del que defenderse; CORS es un permiso que conceder.

Mecanismos de autentificacion soportados

Spring Security soporta muchas formas de autenticar; todas producen el mismo objeto Authentication. Solo cambia como se obtiene la credencial.



Form login

Formulario web con sesion; el clasico de apps con vistas.



JWT / token

Token autocontenido; el estandar para APIs REST sin estado.



LDAP

Contra un directorio corporativo.



HTTP Basic

Usuario y clave en la cabecera; simple, para pruebas o servicios.



OAuth2 / OIDC

Delegar el login en Google, GitHub, etc.



Certificado X.509

Autenticacion por certificado de cliente.

Spring Security 6 y 7: que cambio

Configuracion actual

```
// Estilo moderno (Spring Security 6/7): DSL de lambdas
http
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/auth/**").permitAll()
        .anyRequest().authenticated())
    .httpBasic(Customizer.withDefaults());
```

Ya no existe

WebSecurityConfigurerAdapter (fuera desde la v6).
authorizeRequests(), and(), antMatchers().



Configuracion por beans Defines un SecurityFilterChain como bean, no heredas de una clase base.



DSL de lambdas obligatorio Cada apartado se configura con una lambda; desaparece el encadenado con and().



PathPattern por defecto requestMatchers usa coincidencia de rutas moderna; fuera antMatchers/mvcMatchers.



DE LA TEORIA A LA PRACTICA

Ya tienes el mapa mental

Cadena de filtros, contexto de seguridad, proveedores, UserDetails, codificación de contraseñas, autorización por roles y la elección sin estado. **Cada pieza reaparecerá como código.**

Lo que sigue: la práctica con Spring Security y JWT sobre el proyecto appweb-api.

Gracias